

1.

Clientes(id_cliente, nombre, email, pais, fecha_creacion, ...)

SELECT * FROM Clientes WHERE pais = 'Argentina' OR pais = 'Uzbekistan';

a.

$$F(Clientes) = \frac{n(Clientes)}{B(Clientes)} = \frac{1.200.000}{400.000} = 3$$

Indice de clustering de altura 2 en columna pais.

$$Cost(\sigma_{pais='Argentina' \vee pais='Uzbekistan'}) = Height(I(Pais, Clientes)) + \lceil \frac{n(Clientes)}{V(Pais, Clientes) \cdot F(Clientes)} \rceil$$

Separo en dos casos, primero busco para Argentina:

$$Cost(\sigma_{pais='Argentina'}) = 2 + \lceil \frac{200.000}{3} \rceil = 2 + \lceil 66.666,66 \rceil$$

$$Cost(\sigma_{pais='Argentina'}) = 2 + 66.667 = 66.669$$

Busco para Uzbekistan:

$$Cost(\sigma_{pais='Uzbekistan'}) = Height(I(Pais, Clientes)) + \lceil \frac{n(Clientes)}{V(Pais, Clientes) \cdot F(Clientes)} \rceil$$

$$Cost(\sigma_{pais='Uzbekistan'}) = 2 + \lceil \frac{1.200.000 - 250.000 - 200.000 - 150.000}{80 \cdot 3} \rceil$$

$$Cost(\sigma_{pais='Uzbekistan'}) = 2 + \lceil \frac{600.000}{240} \rceil = 2 + 2.500 = 2.502$$

$$Cost(Total) = 66.669 + 2.502 = 69.171$$

Un File Scan costaria $B(Clientes) = 400.000$ por lo cual conviene utilizar el índice.

b.

$$n(\sigma_{pais='Uzbekistan'}(Clientes)) = \frac{600.000}{80} = 7.500$$

$$n(\sigma_{pais='Argentina' \vee pais='Uzbekistan'}) = n(\sigma_{pais='Argentina'}(Clientes)) + n(\sigma_{pais='Uzbekistan'}(Clientes))$$

$$n(\sigma_{pais='Argentina' \vee pais='Uzbekistan'}) = 200.000 + 7.500 = 207.500$$

2.

- Usuarios(id_usuario, email, direccion, ultimo_login)
- Pedidos(id_pedido, id_usuario, fecha, tipo)

```
SELECT
    fecha
FROM
    Usuarios u, Pedidos p
WHERE
    p.id_usuario = u.id_usuario AND
    u.email = 'immcnabb@nfl.com' AND
    p.tipo = 'Urgente'
```

Indice Usuarios: id_usuario (I1), email (I2).

Indice Pedidos: id_usuario (I3), tipo (I4).

Todos son índices secundarios.

Lo hago de esta forma pues al ser $V(email, Usuarios) = n(Usuarios)$ me va quedar 1 sola fila ahí, por lo cual siguiendo la heurística, realizó las operaciones más restrictivas primero y después aprovecho pipelining. La selección de Pedidos se va hacer en la propia junta con lo que ya tengo en memoria.

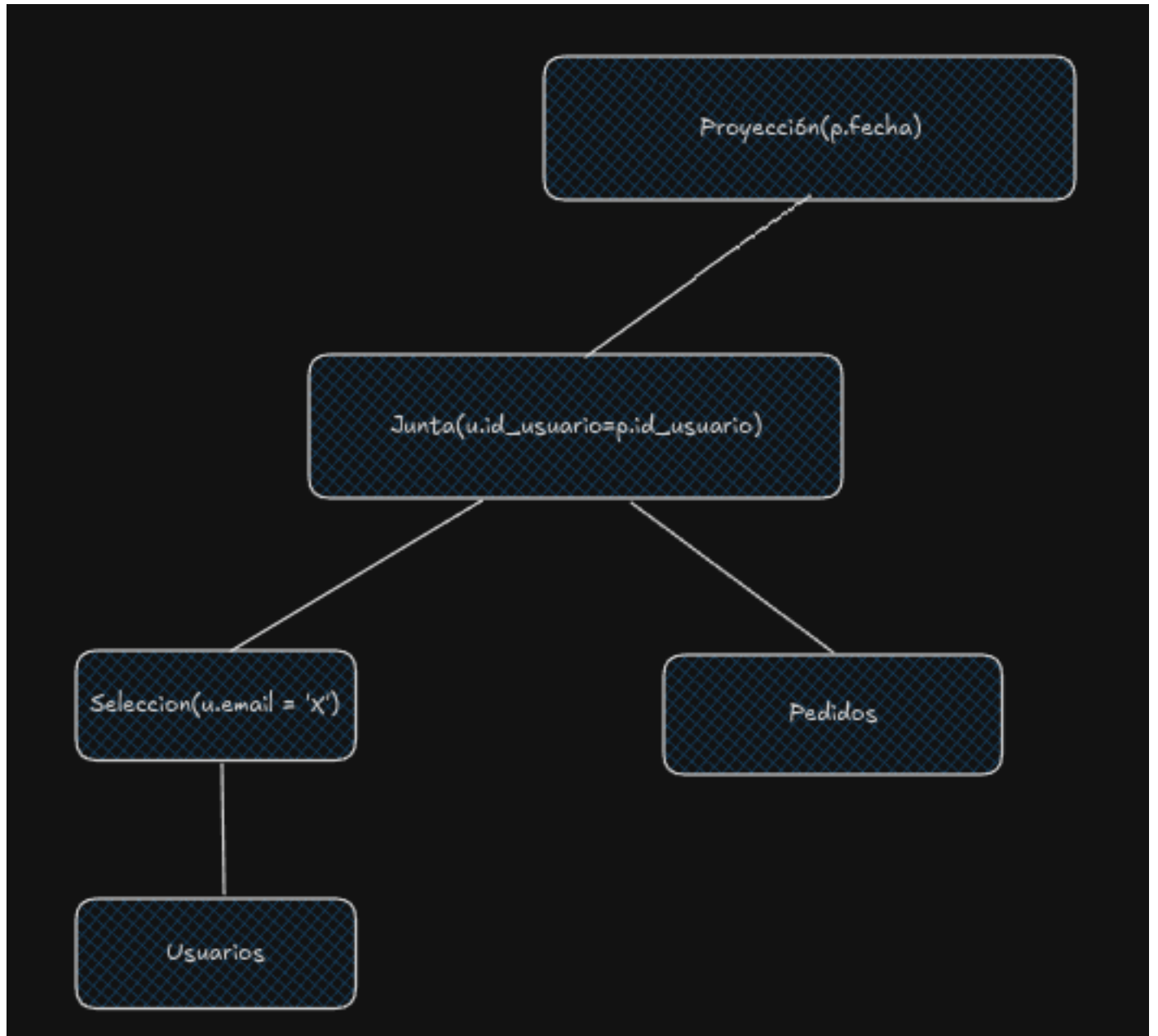


Figure 1: Arbol de consulta

b.

$$Cost(\sigma_{email=X'}(Usuario)) = Height(I(email, Usuarios)) + \lceil \frac{n(Clientes)}{V(email, Clientes)} \rceil = 4 + 1 = 5$$

Para la junta pruebo con loop con unico indice

$$Cost(Usuarios_{\sigma} \bowtie Pedidos) = n(Usuarios_{\sigma}) \cdot \lceil Height(I(id_{usuario}, Pedidos)) + \lceil \frac{n(Pedidos)}{V(id_{usuario}, Pedidos)} \rceil \rceil$$

$$Cost(Usuarios_{\sigma} \bowtie Pedidos) = 1 \cdot \lceil 4 + \lceil \frac{320.000}{80.000} \rceil \rceil = 8$$

Luego como ya dije la seleccion de Pedidos se evalua en memoria cuando se hace la junta, y la proyección como no tiene un DISTINCT tiene un coste 0.

$$Cost(Consulta) = Cost(\sigma_{email=X'}(Usuario)) + Cost(Usuarios_{\sigma} \bowtie Pedidos) = 5 + 8 = 13$$

3.

a.

```
db.papers.aggregate([
  {
    $unwind: "$autores"
  },
  {
    $match: {
      categoria: "Informática"
    }
  },
  {
    $group: {
      _id: "$autores",
      cantidad: {$sum: 1},
      promedio_puntaje: {$avg: "$puntaje"}
    }
  },
  {
    $match: {
      cantidad: {
        $gte: 10
      }
    }
  },
  {
    $project: {
      _id: 0,
      autor: "$_id.autores",
      cantidad: 1,
      promedio_puntaje: 1
    }
  }
])
```

b. Para que sea lo más distribuido posible debería shardearse por id ya que tiene una alta cardinalidad.

No usaria como shard key la categoria ya que no lo hace distribuida, ya que como estoy buscando por categoría estaría buscando todo en un único nodo.

4.

```
MATCH (u1:Usuario)-[p1:PUNTUA]->(pub:Publicacion),
      (u2:Usuario)-[p2:PUNTUA]->(pub:Publicacion)
WHERE u1.username > u2.username AND
```

```

    p1.puntaje >= 8 AND
    p2.puntaje >= 8
WITH u1, u2, COUNT(pub) AS cantidad_publicaciones
WHERE cantidad_publicaciones >= 5
WITH u1, u2
WHERE NOT EXISTS {
    MATCH (u1)-[p3:PUNTUA]->(pub3:Publicacion),
          (u2)-[p4:PUNTUA]->(pub4:Publicacion)
    WHERE pub3.id = pub4.id AND (
        (p3.puntaje >= 8 AND p4.puntaje <= 7)
        OR
        (p4.puntaje >= 8 AND p3.puntaje <= 7)
    )
}
RETURN u1, u2

```

5.

$b_{T_1}; b_{T_2}; b_{T_3}; W_{T_1}(X); R_{T_3}(X); R_{T_2}(Y); R_{T_2}(Z); R_{T_1}(Z); c_{T_1}; R_{T_2}(X); W_{T_3}(Y); c_{T_2}; c_{T_3}$

a.

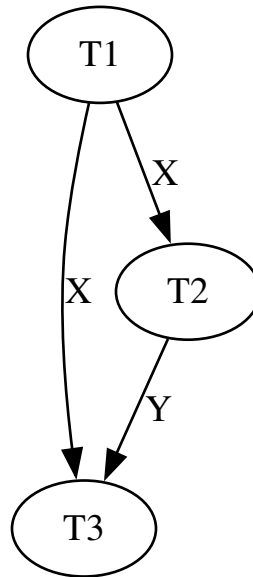


Figure 2: Grafo de precedencias

- b. El solapamiento es serializable porque el grafo de precedencias es un DAG. Una ejecución posible es $T_1; T_2; T_3$
- c. Para que el solapamiento sea recuperable aquellas transacciones que lean un dato modificado previamente por otra transacción, deben commitear después que la transacción “modificadora”. Por lo cual miro las lecturas.
- $R_{T_3}(X)$: T_1 escribe X antes, pero commitea antes esto no rompe nada, por ahora es recuperable.
 - $R_{T_2}(Y)$: nadie modificó a Y antes, no genera problemas.
 - $R_{T_2}(Z)$: idem el item anterior.
 - $R_{T_1}(Z)$: idem el item anterior.
 - $R_{T_2}(X)$: T_1 escribe X antes, pero como T_1 commitea antes que T_2 , por ahora sigue siendo recuperable.

Analizadas todas las lecturas, concluyo que el solapamiento es recuperable.

6.

```

01 (BEGIN, T1)
02 (WRITE, T1, A, 10)
03 (BEGIN, T2)
04 (WRITE, T2, D, 20)
05 (BEGIN, T3)
06 (COMMIT, T2)

```

```

07 (BEGIN CKPT, T1, T3)
08 (WRITE, T3, B, 40)
09 (BEGIN, T4)
10 (COMMIT, T3)
11 (WRITE, T4, C, 17)
12 (END CKPT)
13 (COMMIT, T1)
14 (BEGIN CKPT, T4)
15 (WRITE, T4, B, 30)
16 (COMMIT, T4)

```

Es REDO por lo cual tengo que rehacer las transacciones que ya commitearon. Veo en BEGIN CKPT, T4 pero no su END, por lo cual busco el siguiente BEGIN CKPT. Estoy seguro que arriba del BEGIN CKPT de la linea 07 está flusheado a disco, lo de abajo no.

Commitearon sin flush (T1, T3, T4)

Voy hasta la linea 01. (el begin de la transaccion mas vieja que)

```

A <- 10
B <- 40
C <- 17
B <- 30

```

Como todas commitearon no agrego nada al log.

Flusheo todo a disco.

1.

```

SELECT
  a.id_antro, nombre
FROM
  antros a
  INNER JOIN disponibilidad d USING(id_antro)
WHERE
  (a.provincia = 'Catamarca' OR a.provincia = 'La Rioja') AND
  a.capacidad >= 50.000 AND
  (d.fecha BETWEEN '2025-01-06' AND '2025-01-10')

```

Para calcular el costo de esta consulta voy a ver por donde me conviene arrancar. Tengo dos opciones para aprovechar pipelining, hacer la seleccion por capacidad o la seleccion segun la fecha. Voy a priorizar aquella selección que de como resultado menos rows.

El enunciado me dice que solo el 10% de los antros tiene capacidad para al menos 50.000 personas, por lo cual miro:

$$n(\sigma_{capacidad \geq 50.000 \wedge (provincia = 'x' \vee provincia = 'y')}(Antros)) = \frac{0,1 \cdot 2 \cdot 10.000}{20} = \frac{2.000}{20} = 100$$

Ya a ojo estoy casi seguro que conviene esta selección primero, pero hago la cuenta si hiciera la selección primero del between

$$n(\sigma_{'2025-01-06' \leq fecha \leq '2025-01-10'}(Disponibilidad)) = \lceil \frac{5}{100} \cdot \frac{100.000}{10.000} \rceil = 0,5$$

Empiezo ahora sí a calcular el costo.

Uso el indice de provincia y dsp el chequeo de la capacidad de la capacidad lo hago en memoria

$$Cost(\sigma_{provincia = 'x' \vee provincia = 'y'}(Antros)) = Height(I(Provincia, Antros)) + \lceil \frac{n(Antros)}{V(Provincia, Antros)} \rceil$$

$$Cost(\sigma_{provincia = 'x' \vee provincia = 'y'}(Antros)) = 2 \cdot [2 + \lceil \frac{10.000}{20} \rceil] = 2 \cdot 502 = 1.004$$

Calculo el costo de la junta. Pruebo con loop con único índice. Ya tengo en memoria (usando pipelining) a los bloques de Antros por lo cual esa parte del costo de la junta será 0

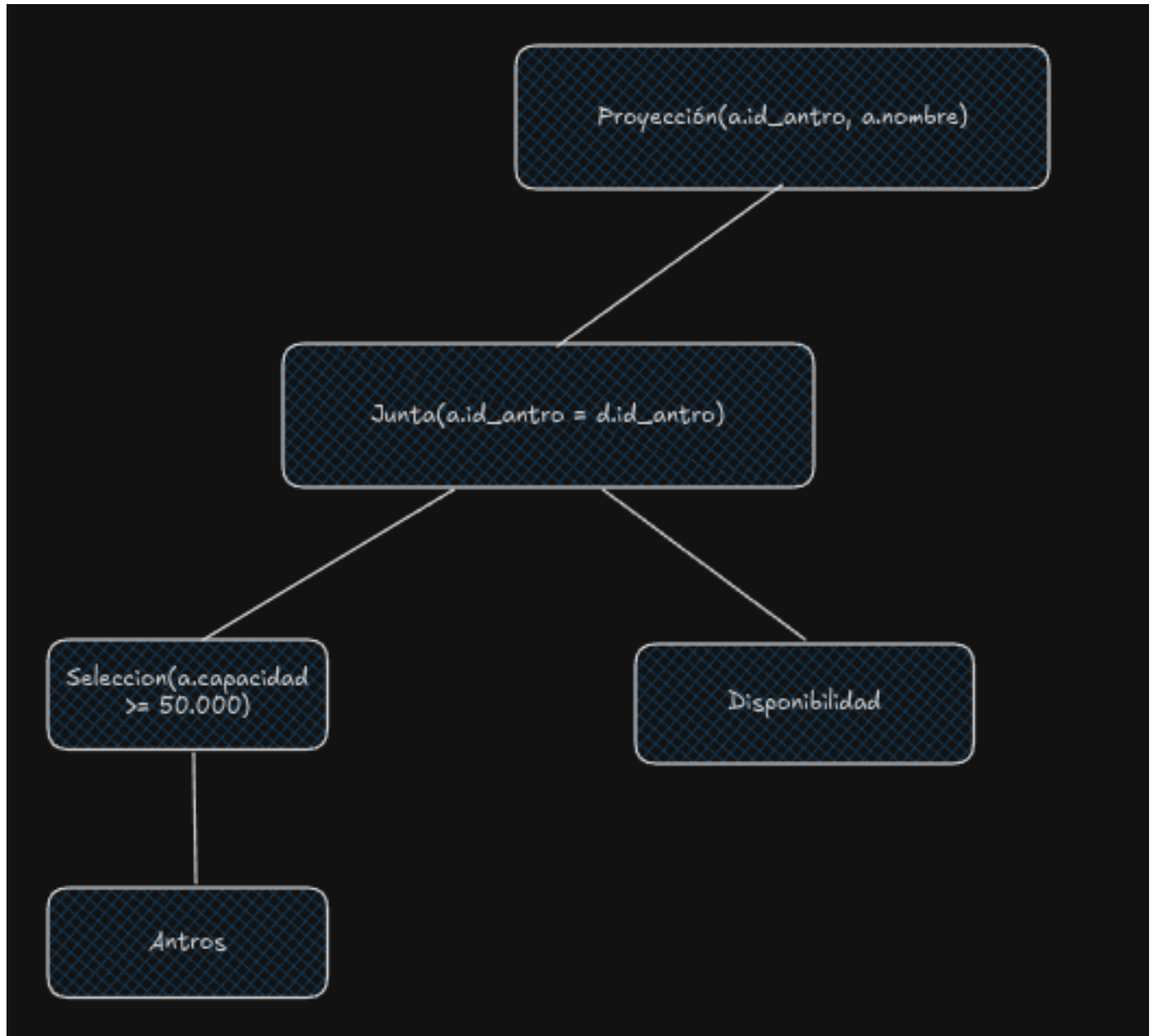


Figure 3: Árbol de consulta

$$Cost(Antros \bowtie Disponibilidad) = n(Antros_{\sigma}) \cdot [Height(I(id_antro, Disponibilidad)) + \lceil \frac{n(Disponibilidad)}{V(id_antro, Disponibilidad)} \rceil]$$

$$Cost(Antros \bowtie Disponibilidad) = 100 \cdot [4 + \lceil \frac{100.000}{10.000} \rceil] = 1.400$$

Luego la seleccion de la fecha también se hace en memoria, y como la proyección no tiene DISTINCT el costo es 0 dejando un coste final de

$$Cost(Consulta) = Cost(\sigma_{provincia='x' \vee provincia='y'}(Antros)) + Cost(Antros \bowtie Disponibilidad) = 1.004 + 1.400 = 2.404$$

Como estime 50 filas para la seleccion completa de Antros y 5.000 para la de Disponibilidad las filas finales serán

$$n(Consulta) = 100 \cdot \frac{5}{100} \cdot 10 = 50$$

2.

- GenerosFavoritos(id_usuario, id_genero)
- Juegos(id_juego, nombre, id_genero)

Usando Hash Grace

$$Cost(GenerosFavoritos \bowtie Juegos) = 3 \cdot (B(GenerosFavoritos) + B(Juegos)) = 3 \cdot 6.000 = 18.000$$

Calculo ahora la cantidad minima de memoria

En Hash Grace se debe cumplir:

$$P \leq M - 1$$

y

$$\min(\lceil \frac{B(GenerosFavoritos)}{P} \rceil; \lceil \frac{B(Juegos)}{P} \rceil) \leq M - 2$$

Reemplazando e igualando para hacer el “peor caso”:

$$\begin{aligned} \frac{1.000}{M-1} &= M-2 \rightarrow 1000 = (M-2) \cdot (M-1) \\ 1000 &= M^2 - 3M + 2 \rightarrow M^2 - 3M - 998 = 0 \end{aligned}$$

Usando Bhaskara obtengo:

$$M1 = 33, 12; M2 = -30, 12$$

Tengo que usar un valor real positivo y redondeando para arriba obtengo entonces que necesito $M = 34$

3.

```
MATCH(c1:Clase{tipo:"Presencial"})
WITH COUNT(c1) AS cantidad_clases_presenciales

MATCH(a1:Alumno)-[asistencias:ASISTIO{tipo:"Presencial"}]-(c2:Clase{tipo:"Presencial"})
WITH a1, cantidad_clases_presenciales, COUNT(c2) AS asistencias_alumno

WHERE a1.nota < 4 AND
      cantidad_clases_presenciales = asistencias_alumno
RETURN a1.padron, a1.nombre
```

4.

```

db.premios.aggregate([
  {
    $match: {
      año: {$gte: 2010}
    }
  },
  {
    $unwind: "$ganadores"
  },
  {
    $group: {
      _id: {
        plato: "$ganadores.plato",
        año: "$año"
      }
    }
  },
  {
    $group: {
      _id: {
        plato: "$_id.plato"
      }
      cantidad_años: {$sum: 1}
    }
  },
  {
    $match: {
      cantidad_años: {$gte: 3}
    }
  },
  {
    $project: {
      _id: 0,
      nombre_plato: "$_id.plato",
      años_ganando: "$cantidad_años"
    }
  }
])

```

5.

$$b_{T_1}; R_{T_1}(X); b_{T_2}; R_{T_2}(X); W_{T_2}(Y); b_{T_3}; W_{T_3}(Z); R_{T_1}(Z); W_{T_3}(X); R_{T_1}(Y)$$

a.

Como el grafo no es un DAG, el solapamiento no es serializable.

b. Para que el solapamiento sea serializable las transacciones que lean un item modificado previamente por otra transacción, deben commitear después que la transacción “modificadora”. Miro las lecturas que pueden generar problemas (ignoro las primeras que no hay ninguna escritura previa):

- T_1 debe commitear después que T_3 por el par $W_{T_3}(Z); R_{T_1}(Z)$
- T_1 debe commitear después que T_2 por el par $W_{T_2}(Y); R_{T_1}(Y)$

Propongo entonces como posible solución $c_{T_2}; c_{T_3}; c_{T_1}$

c. El solapamiento con los commits agregado permite el rollback en cascada pues para evitarlos los commits deberían estar previo a las lecturas “conflictivas”, y como se pide agregarlas todas al final, no hay forma de evitar esta permisión.

6.

```

01 (BEGIN, T1)
02 (WRITE T1, A, 10)
03 (BEGIN, T2)
04 (WRITE T1, B, 20)

```

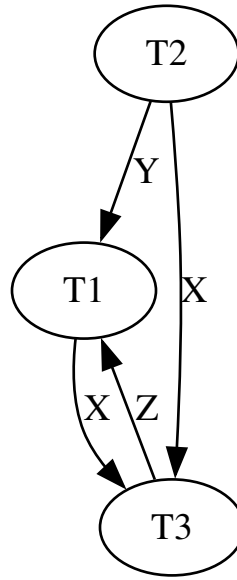



Figure 4: Grafo de precedencias

```

05 (COMMIT, T1)
06 (WRITE T2, B, 30)
07 (BEGIN CKPT, T2)
08 (BEGIN, T3)
09 (WRITE T3, C, 40)
10 (COMMIT, T2)
11 (BEGIN, T4)
12 (WRITE T4, D, 50)
13 (END, CKPT)
14 (COMMIT, T4)

```

Tengo que rehacer las transacciones commiteadas (T2, T4). Veo un CKPT con su END donde solo tengo T2. Busco entonces el inicio de T2

Llego a la linea 03 y arranco a rehacer

```

B <- 30
D <- 50

```

Agrego el correspondiente (*ABORT*, T3) pues no commiteó. Flusheo todo a disco!

1.

Tengo dos opciones: o realizar la seleccion de clientes primero, usar pipelining, juntar y luego hacer la selección de prestamos; o realizar la seleccion de prestamos primero usar pipelining y juntar y luego hacer la selección de clientes.

Estimo entonces cuantas bloques me va dejar cada selección de hacerla primero. Asumo distro uniforme de aquellas variables que no tengo datos.

$$F(Clientes) = \frac{n(Clientes)}{B(Clientes)} = \frac{100.000}{20.000} = 5$$

$$n(\sigma_{provincia='Catamarca' \wedge tipo='Premium'}(Clientes)) = \lceil \frac{n(Clientes)}{V(provincia, Clientes) \cdot V(tipo, Clientes)} \rceil = \lceil \frac{100.000}{20 \cdot 4} \rceil = 1.250$$

$$B(\sigma_{provincia='Catamarca' \wedge tipo='Premium'}(Clientes)) = \lceil \frac{n_{\sigma}(Clientes)}{F(Clientes)} \rceil = \lceil \frac{1.250}{5} \rceil = 250$$

La selección de la fecha la hago en la propia junta en memoria, no genera un costo extra.

Calculo el costo de la primera selección. No son de clustering, hago una busqueda por el indice que me da menor costo y la otra restricción la chequeo en memoria. Si bien la altura de provincia es mayor, tiene mayor variabilidad asique uso ese:

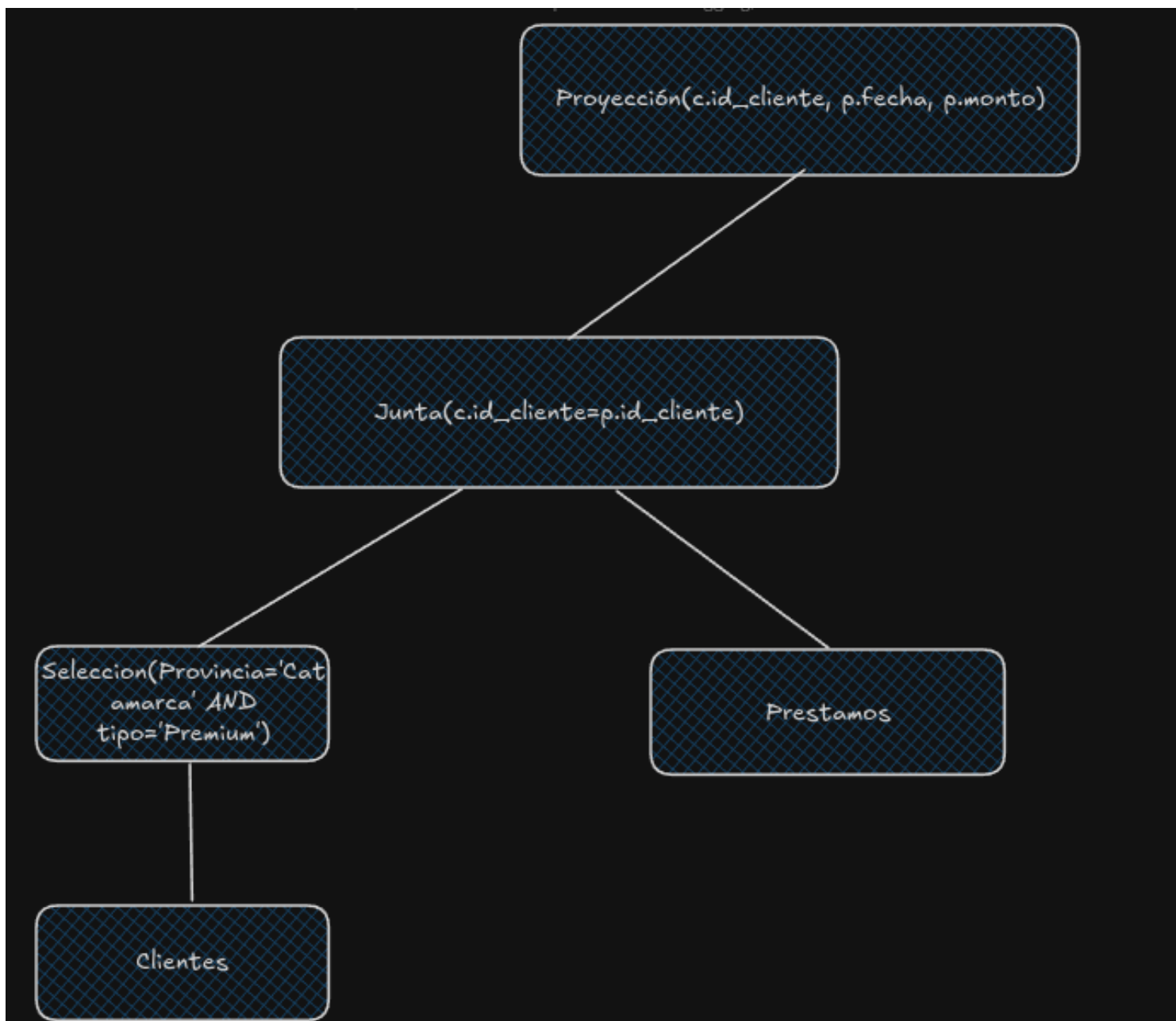


Figure 5: Árbol de consulta

$$Cost(\sigma_{provincia='Catamarca' \wedge tipo='Premium'}(Clientes)) = Height(I(Provincia, Clientes)) + \lceil \frac{n(Clientes)}{V(Provincia, Clientes)} \rceil$$

$$Cost(\sigma_{provincia='Catamarca' \wedge tipo='Premium'}(Clientes)) = 2 + \lceil \frac{100.000}{20} \rceil = 2 + 5.000 = 5.002$$

Ahora busco el costo de la junta, como ya tengo cargado la tabla de clientes (uso pipelining), no sumo el coste inicial. Pruebo con junta por loop con único índice usando el índice de Préstamos:

$$Cost(Clientes_{\sigma} \bowtie Prestamos) = n(Clientes_{\sigma}) \cdot \lceil Height(I(id_cliente, Prestamos)) + \lceil \frac{n(Prestamos)}{V(id_cliente, Prestamos)} \rceil \rceil$$

$$Cost(Clientes_{\sigma} \bowtie Prestamos) = 1.250 \cdot \lceil 3 + \lceil \frac{100.000}{10.000} \rceil \rceil = 1250 \cdot 13 = 16.250$$

Veo que es la mejor opcion de junta porque en el caso de loops anidados por bloque tendría 3 veces los bloques de Prestamos (Cost=30.000). Hash Grace tendría $3 \cdot [B(Clientes) + B(Prestamos)]$ lo cual claramente es peor, y en el caso de Sort Merge por lo menos tendría $B(Clientes) + B(Prestamos)$ lo cual ya es peor que lo calculado.

Concluyo entonces que el costo total de la consulta es:

$$Cost(Consulta) = 5.002 + 16.250 = 21.252$$

La proyección y el resto de restricciones que se imponen en las selecciones se hace aprovechando pipelining/que ya tenemos cosas en memoria.

Estimo ahora sí la cantida de filas resultantes de la consulta. Ya se que de la primera selección obtengo 1.250 filas y también que de prestamos tengo $0,05 \cdot 100.000 = 5.000$ filas.

$$n(Clientes \bowtie Prestamos) = \frac{n_{\sigma}(Clientes) \cdot n_{\sigma}(Prestamos)}{\max(V(id_cliente, Clientes), V(id_cliente, Prestamos))} = \frac{6.250.000}{10.000} = 625$$

$$n(Clientes \bowtie Prestamos) = \frac{1.250 \cdot 5.000}{\max(1.250, 5.000)} = \frac{6.250.000}{5.000} = 625$$

2.

Fulki se que tengo $70.000 \cdot 0,05$ (chances de que ponga un 10) $\rightarrow 3500$

Estimo la cantidad que va tener D.J. Ann

$$n(\sigma_{p.usuario='D.J. Ann'}(Puntuaciones)) = 1.000.000 - 100.000 - 70.000 - 50.000 = 780.000$$

Divido ahora por la variabilidad

$$\frac{780.000}{100.000 - 3(conocidos)} = 7,8 = 8$$

Ademas se que ponen un 10 en el 5% de sus puntuaciones por lo cual hago

$$0.05 \cdot 8 < 1 = 1 \text{ lo aproximo a } 1$$

Para estimar la cantidad de filas, el peor caso va ser que la $V(id_lugar, Puntuaciones)$ sea el mismo que el de lugar asique uso eso como max

$$n(Puntuaciones \bowtie Lugares) = \frac{3.501 \cdot 50.000}{50.000} = 3.501$$

Para estimar ahora la cantidad de bloques:

$$F(Puntuaciones) = \frac{n(Puntuaciones)}{B(puntuaciones)} = \frac{1.000.000}{100.000} = 10$$

$$F(Lugares) = \frac{n(Lugares)}{B(Lugares)} = \frac{50.000}{10.000} = 5$$

$$F(Puntuaciones \bowtie Lugares) = \lfloor \frac{1}{\frac{1}{F(Puntuaciones)} + \frac{1}{F(Lugares)}} \rfloor$$

$$F(Puntuaciones \bowtie Lugares) = \lfloor \frac{1}{\frac{1}{10} + \frac{1}{5}} \rfloor = \lfloor 3,33 \rfloor = 3$$

$$B(Puntuaciones \bowtie Lugares) = \frac{n(Puntuaciones \bowtie Lugares)}{F(Puntuaciones \bowtie Lugares)}$$

$$B(Puntuaciones \bowtie Lugares) = \frac{3.501}{3} = 1167$$

3.

```
MATCH(j:Jugador)-[e1:ELIGIO]->(:Elemento)-[:GANA_A]-(:Elemento)<-[e2:ELIGIO]-(:Jugador)
WHERE e1.partido = e2.partido
WITH j, COUNT(*) AS partidas_ganadas
WHERE partidas_ganadas > 100
```

```
MATCH (j)-[e3:ELIGIO]->(:Elemento)<-[:GANA_A]-(:Elemento)<-[e4:ELIGIO]-(:Jugador)
WHERE e3.partido = e4.partido
WITH j, COUNT(*) AS partidas_perdidas
WHERE partidas_perdidas >= 1 AND partidas_perdidas <= 10
```

```
MATCH(j)-[e5:ELIGIO]->(:Elemento)-[:GANA_A]-(:Elemento)<-[e6:ELIGIO]-(:Jugador{nombre:'Jules'})
WHERE e5.partido = e6.partido
WITH j, COUNT(*) AS total_partidas_jules
```

```
MATCH(j)-[e7:ELIGIO]->(:Elemento)<-[:GANA_A]-(:Elemento)<-[e8:ELIGIO]-(:Jugador{nombre:'Jules'})
WHERE e7.partido = e8.partido
WITH j, total_partidas_jules, COUNT(*) as partidas_perdidas_jules
WHERE total_partidas_jules = partidas_perdidas_jules
```

```
RETURN j
```

4.

```
db.materias.aggregate([
  {
    $unwind: "$notas"
  },
  {
    $match: {
      "$notas.notaFinal": {$gte: 4}
    }
  },
  {
    $group: {
      _id: "$materia",
      promedio_materia: {$avg: "$notas.notaFinal"},
      departamento: { $first: "$departamento" }
    }
  },
  {
    $match: {
      promedio_materia: {$gte: 8}
    }
  },
  {
    $group: {
      _id: "$departamento",
      cant_mate_mayor_8: {$sum: 1}
    }
  }
])
```

```

    }
  },
  {
    $match: {
      cant_mate_mayor_8: {$gt: 0}
    }
  },
  {
    $project: {
      _id: 0,
      departamento: "$_id",
      cant_mate_mayor_8: 1
    }
  }
}
])
5.

```

$b_{T_1}; b_{T_2}; R_{T_1}(X); R_{T_2}(X); b_{T_3}; W_{T_3}(Y); W_{T_3}(Z); W_{T_1}(Y); W_{T_2}(X); R_{T_2}(Z)$

a. Creo grafo de precedencias para ver si el solapamiento es serializable:

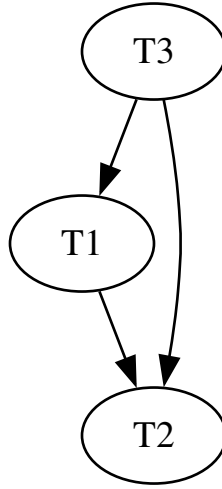


Figure 6: Grafo de precedencias

Como el grafo es un DAG, entonces el solapamiento es serializable.

Propongo un orden de transacciones equivalente:

$$T_3 \rightarrow T_1 \rightarrow T_2$$

b. Para que el solapamiento sea recuperable aquellas transacciones que leen un item modificado previamente por otra transacción, deben commitear después que la transacción “modificadora”. Busco entonces las lecturas:

las primeras dos lecturas se realizan antes de que se haga una modificacion asique ni siquiera las analizo.

Luego solo tengo

$R_{T_2}(Z)$: T_3 modifiko previamente a $T_2 \rightarrow T_3$ tiene que commitear antes que T_2

Agrego al final entonces una posible forma de agregar los commits

$$C_{T_1}; C_{T_3}; C_{T_2}$$

c. Si la transacción T_3 se realiza antes que la transacción T_2 , y de no haber un commit en el medio, T_2 lee Z que fue modificado por T_3 por lo cual el solapamiento permite rollbacks en cascada.

6.

```

01 (BEGIN, T1);
02 (WRITE T1, A, 1);
03 (WRITE T1, B, 2);
04 (BEGIN, T2);
05 (COMMIT, T1);
06 (WRITE T2, B, 3);
07 (BEGIN, T3);
08 (BEGIN CKPT, T2, T3);
09 (WRITE T3, C, 4);
10 (COMMIT, T2);
11 (BEGIN, T4);
12 (WRITE T4, D, 5);
13 (COMMIT, T4);

```

Tengo que deshacer las transacciones no commiteadas. Tengo un BEGIN CKPT sin END por lo cual debo deshacer la mas vieja de esas transacciones

Llego hasta la linea 04.

Veo no commiteadas T_3 .

$$C \leftarrow 4$$

Agrego al final del log

$$(ABORT, T_3)$$

flusho todo a disco